Sales & Order Management Analytics — SQL

FINAL — Copyable SQL Version

Sales & Order Management Analytics

Complete SQL Query Book — 20-Record Database

Coverage: 20 Customers, 20 Products, 20 Orders, related Order Items, and 20 Payments. This book contains actual SQL queries for the requested database, SQL concepts, and business analysis.

1. Database & Tables

```
CREATE DATABASE sales_order_analytics;

USE sales_order_analytics;

CREATE TABLE customers (

customer_id INT PRIMARY KEY,

name VARCHAR(100) NOT NULL,

email VARCHAR(150) NOT NULL UNIQUE,

city VARCHAR(80) NOT NULL,

signup_date DATE NOT NULL

);

CREATE TABLE products (

product_id INT PRIMARY KEY,

product_name VARCHAR(120) NOT NULL,

category VARCHAR(80) NOT NULL,

price DECIMAL(10,2) NOT NULL CHECK (price > 0),

stock INT NOT NULL CHECK (stock >= 0)

);

CREATE TABLE orders (

order_id INT PRIMARY KEY,

customer_id INT NOT NULL,

order_date DATE NOT NULL,

status VARCHAR(20) NOT NULL CHECK (status IN ('COMPLETED','PENDING','CANCELLED')),

total_amount DECIMAL(12,2) NOT NULL CHECK (total_amount >= 0),

FOREIGN KEY (customer_id) REFERENCES customers(customer_id)

);

CREATE TABLE order_items (

order_item_id INT PRIMARY KEY,

order_id INT NOT NULL,

product_id INT NOT NULL,

quantity INT NOT NULL CHECK (quantity > 0),
```

```
unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),
FOREIGN KEY (order_id) REFERENCES orders(order_id),
FOREIGN KEY (product_id) REFERENCES products(product_id)
);
CREATE TABLE payments (
payment_id INT PRIMARY KEY,
order_id INT NOT NULL UNIQUE,
payment_date DATE NOT NULL,
amount DECIMAL(12,2) NOT NULL CHECK (amount >= 0),
payment_status VARCHAR(20) NOT NULL CHECK (payment_status IN ('SUCCESS','PENDING','FAILED')),
FOREIGN KEY (order_id) REFERENCES orders(order_id)
);
```

2. DML — 20 Customers

```
INSERT INTO customers (customer_id,name,email,city,signup_date) VALUES
(1,'Aarav Sharma','aarav@example.com','Delhi','2024-01-10'),
(2,'Pooja Desai','pooja@example.com','Mumbai','2024-01-12'),
(3,'Meera Yadav','meera@example.com','Bangalore','2024-01-15'),
(4,'Priya Patel','priya@example.com','Pune','2024-01-18'),
(5,'Riya Singh','riya@example.com','Hyderabad','2024-01-20'),
(6,'Rahul Kumar','rahul@example.com','Chennai','2024-01-22'),
(7,'Ananya Rao','ananya@example.com','Kolkata','2024-01-25'),
(8,'Karan Mehta','karan@example.com','Delhi','2024-01-27'),
(9,'Sneha Iyer','sneha@example.com','Mumbai','2024-02-01'),
(10,'Vikram Nair','vikram@example.com','Pune','2024-02-04'),
(11,'Kavya Reddy','kavya@example.com','Hyderabad','2024-02-08'),
(12,'Arjun Shah','arjun@example.com','Bangalore','2024-02-10'),
(13,'Neha Gupta','neha@example.com','Delhi','2024-02-12'),
(14,'Rohan Das','rohan@example.com','Kolkata','2024-02-15'),
(15,'Isha Verma','isha@example.com','Mumbai','2024-02-18'),
(16,'Aditya Joshi','aditya@example.com','Chennai','2024-02-20'),
(17,'Nisha Rao','nisha@example.com','Pune','2024-02-22'),
(18,'Sahil Khan','sahil@example.com','Hyderabad','2024-02-25'),
(19,'Tanvi Jain','tanvi@example.com','Bangalore','2024-02-27'),
(20,'Manish Roy','manish@example.com','Delhi','2024-03-01');
```

3. DML — 20 Products

```
INSERT INTO products (product_id,product_name,category,price,stock) VALUES
```

```
(1,'Smartphone','Electronics',18000,50),
(2,'Laptop','Electronics',32000,25),
(3,'Headphones','Electronics',2500,80),
(4,'Smartwatch','Electronics',5500,40),
(5,'Keyboard','Electronics',1800,60),
(6,'Mouse','Electronics',900,100),
(7,'T-Shirt','Clothing',750,120),
(8,'Jeans','Clothing',1800,70),
(9,'Shoes','Clothing',3500,45),
(10,'Jacket','Clothing',4200,35),
(11,'Table Lamp','Home Appliances',1200,55),
(12,'Mixer Grinder','Home Appliances',4500,30),
(13,'Air Fryer','Home Appliances',6500,20),
(14,'Cookware Set','Home Appliances',3800,25),
(15,'Novel','Books',500,90),
(16,'Textbook','Books',900,75),
(17,'Backpack','Accessories',1400,65),
(18,'Wallet','Accessories',800,85),
(19,'Water Bottle','Accessories',600,110),
(20,'Belt','Accessories',700,95);
```

4. DML — 20 Orders

```
INSERT INTO orders (order_id,customer_id,order_date,status,total_amount) VALUES
(1,1,'2024-01-05','COMPLETED',20000),
(2,2,'2024-01-12','COMPLETED',32000),
(3,3,'2024-02-03','COMPLETED',18000),
(4,4,'2024-02-15','PENDING',7500),
(5,5,'2024-03-02','COMPLETED',12500),
(6,1,'2024-03-10','COMPLETED',5500),
(7,2,'2024-03-18','CANCELLED',1800),
(8,6,'2024-04-04','COMPLETED',4200),
(9,7,'2024-04-15','COMPLETED',6500),
(10,8,'2024-05-02','COMPLETED',3800),
(11,9,'2024-05-11','PENDING',9000),
(12,10,'2024-05-20','COMPLETED',7000),
(13,11,'2024-06-01','COMPLETED',18000),
(14,3,'2024-06-10','COMPLETED',2500),
(15,4,'2024-06-21','COMPLETED',4500),
```

```
(16,12,'2024-07-03','COMPLETED',32000),

(17,13,'2024-07-12','CANCELLED',1400),

(18,14,'2024-07-20','COMPLETED',5500),

(19,15,'2024-07-25','COMPLETED',3000),

(20,1,'2024-07-30','COMPLETED',9000);
```

5. Related Order Items

```
INSERT INTO order_items (order_item_id,order_id,product_id,quantity,unit_price) VALUES

(1,1,1,1,18000),(2,1,3,1,2500),

(3,2,2,1,32000),

(4,3,1,1,18000),

(5,4,7,2,750),(6,4,18,1,800),

(7,5,4,1,5500),(8,5,9,2,3500),

(9,6,4,1,5500),

(10,7,5,1,1800),

(11,8,10,1,4200),

(12,9,13,1,6500),

(13,10,14,1,3800),

(14,11,6,10,900),

(15,12,20,10,700),

(16,13,1,1,18000),

(17,14,3,1,2500),

(18,15,12,1,4500),

(19,16,2,1,32000),

(20,17,17,1,1400),

(21,18,4,1,5500),

(22,19,19,5,600),

(23,20,6,10,900);
```

6. 20 Payments

```
INSERT INTO payments (payment_id,order_id,payment_date,amount,payment_status) VALUES

(1,1,'2024-01-05',20000,'SUCCESS'),

(2,2,'2024-01-12',32000,'SUCCESS'),

(3,3,'2024-02-03',18000,'SUCCESS'),

(4,4,'2024-02-15',7500,'PENDING'),

(5,5,'2024-03-02',12500,'SUCCESS'),

(6,6,'2024-03-10',5500,'SUCCESS'),

(7,7,'2024-03-18',0,'FAILED'),
```

```
(8,8,'2024-04-04',4200,'SUCCESS'),

(9,9,'2024-04-15',6500,'SUCCESS'),

(10,10,'2024-05-02',3800,'SUCCESS'),

(11,11,'2024-05-11',9000,'PENDING'),

(12,12,'2024-05-20',7000,'SUCCESS'),

(13,13,'2024-06-01',18000,'SUCCESS'),

(14,14,'2024-06-10',2500,'SUCCESS'),

(15,15,'2024-06-21',4500,'SUCCESS'),

(16,16,'2024-07-03',32000,'SUCCESS'),

(17,17,'2024-07-12',0,'FAILED'),

(18,18,'2024-07-20',5500,'SUCCESS'),

(19,19,'2024-07-25',3000,'SUCCESS'),

(20,20,'2024-07-30',9000,'SUCCESS');
```

7. CRUD

```
-- SELECT

SELECT * FROM customers;

-- INSERT

INSERT INTO customers VALUES (21,'Test User','test@example.com','Delhi','2024-03-05');

-- UPDATE

UPDATE customers SET city='Mumbai' WHERE customer_id=21;

-- DELETE

DELETE FROM customers WHERE customer_id=21;
```

30. DML — Orders (INSERT / SELECT / UPDATE / DELETE)

```
-- INSERT

INSERT INTO orders

(order_id, customer_id, order_date, status, total_amount)

VALUES

(21, 1, '2024-08-01', 'COMPLETED', 15000.00);

-- SELECT

SELECT * FROM orders;

-- UPDATE

UPDATE orders

SET status = 'COMPLETED'

WHERE order_id = 21;

-- DELETE

DELETE FROM orders
```

```
WHERE order_id = 21;
```

8. JOINs

```
-- INNER JOIN

SELECT o.order_id,c.name,o.total_amount

FROM orders o INNER JOIN customers c ON o.customer_id=c.customer_id;

-- LEFT JOIN

SELECT c.customer_id,c.name,o.order_id

FROM customers c LEFT JOIN orders o ON c.customer_id=o.customer_id;

-- MULTI-TABLE JOIN

SELECT o.order_id,c.name,p.product_name,oi.quantity,oi.unit_price

FROM orders o

JOIN customers c ON o.customer_id=c.customer_id

JOIN order_items oi ON o.order_id=oi.order_id

JOIN products p ON oi.product_id=p.product_id;
```

9. Aggregations / GROUP BY / HAVING

```
SELECT SUM(total_amount) AS total_revenue FROM orders;

SELECT COUNT(*) AS total_orders FROM orders;

SELECT AVG(total_amount) AS average_order_value FROM orders;

SELECT MIN(total_amount) AS minimum_order FROM orders;

SELECT MAX(total_amount) AS maximum_order FROM orders;

SELECT c.city,SUM(o.total_amount) AS sales

FROM orders o JOIN customers c ON o.customer_id=c.customer_id

GROUP BY c.city;

SELECT c.city,SUM(o.total_amount) AS sales

FROM orders o JOIN customers c ON o.customer_id=c.customer_id

GROUP BY c.city

HAVING SUM(o.total_amount)>10000;
```

10. Subquery

```
SELECT name

FROM customers

WHERE customer_id IN (

SELECT customer_id FROM orders

GROUP BY customer_id

HAVING COUNT(*) > 1

);
```

11. CTE

```
WITH monthly_sales AS (

SELECT DATE_FORMAT(order_date,'%Y-%m') AS month,

SUM(total_amount) AS sales

FROM orders

GROUP BY DATE_FORMAT(order_date,'%Y-%m')

)

SELECT * FROM monthly_sales ORDER BY month;
```

12. CASE

```
SELECT order_id,total_amount,

CASE

WHEN total_amount >= 20000 THEN 'High Value'

WHEN total_amount >= 10000 THEN 'Medium Value'

ELSE 'Low Value'

END AS order_value

FROM orders;
```

13. Window Functions & Ranking

```
SELECT order_id,customer_id,total_amount,

ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY total_amount DESC) AS order_rank

FROM orders;

SELECT c.name,SUM(o.total_amount) AS sales,

RANK() OVER (ORDER BY SUM(o.total_amount) DESC) AS customer_rank

FROM customers c JOIN orders o ON c.customer_id=o.customer_id

GROUP BY c.customer_id,c.name;
```

14. Running Total

```
SELECT order_date,order_id,total_amount,

SUM(total_amount) OVER (ORDER BY order_date,order_id) AS running_total

FROM orders;
```

15. Month-over-Month

```
WITH m AS (

SELECT DATE_FORMAT(order_date,'%Y-%m') month,SUM(total_amount) sales

FROM orders GROUP BY DATE_FORMAT(order_date,'%Y-%m')

)

SELECT month,sales,

LAG(sales) OVER (ORDER BY month) previous_month,

sales-LAG(sales) OVER (ORDER BY month) AS change_amount,
```

```
ROUND((sales-LAG(sales) OVER (ORDER BY month))
/NULLIF(LAG(sales) OVER (ORDER BY month),0)*100,2) AS change_percent
FROM m ORDER BY month;
```

16. View

```
CREATE OR REPLACE VIEW customer_sales AS
SELECT c.customer_id,c.name,c.city,SUM(o.total_amount) sales
FROM customers c JOIN orders o ON c.customer_id=o.customer_id
GROUP BY c.customer_id,c.name,c.city;
SELECT * FROM customer_sales;
```

17. Indexes & EXPLAIN

```
CREATE INDEX idx_orders_customer ON orders(customer_id);
CREATE INDEX idx_orders_date ON orders(order_date);
CREATE INDEX idx_order_items_product ON order_items(product_id);
EXPLAIN SELECT o.order_id,c.name,o.total_amount
FROM orders o JOIN customers c ON o.customer_id=c.customer_id
WHERE o.order_date >= '2024-06-01';
```

18. Transaction

```
START TRANSACTION;
UPDATE products SET stock=stock-1
WHERE product_id=1 AND stock>0;
INSERT INTO order_items(order_item_id,order_id,product_id,quantity,unit_price)
VALUES (24,20,1,1,18000);
COMMIT;
-- If something goes wrong:
-- ROLLBACK;
```

19. Inventory Analysis

```
SELECT product_id,product_name,stock,
CASE
WHEN stock=0 THEN 'Out of Stock'
WHEN stock<20 THEN 'Low Stock'
ELSE 'In Stock'
END AS inventory_status
FROM products;
SELECT p.product_name,SUM(oi.quantity) AS units_sold
FROM products p JOIN order_items oi ON p.product_id=oi.product_id
GROUP BY p.product_id,p.product_name
```

```
ORDER BY units_sold DESC;
```

20. Business Analysis — Core KPIs

```
-- Total Sales / Revenue

SELECT SUM(total_amount) AS total_sales FROM orders;

-- Total Orders

SELECT COUNT(*) AS total_orders FROM orders;

-- Total Customers

SELECT COUNT(*) AS total_customers FROM customers;

-- Total Products

SELECT COUNT(*) AS total_products FROM products;

-- Average Order Value

SELECT AVG(total_amount) AS average_order_value FROM orders;
```

21. Sales by City

```
SELECT c.city,SUM(o.total_amount) AS sales

FROM customers c JOIN orders o ON c.customer_id=o.customer_id

GROUP BY c.city ORDER BY sales DESC;
```

22. Monthly Sales Trend

```
SELECT DATE_FORMAT(order_date,'%Y-%m') AS month,

SUM(total_amount) AS monthly_sales

FROM orders

GROUP BY DATE_FORMAT(order_date,'%Y-%m')

ORDER BY month;
```

23. Top Products by Sales

```
SELECT p.product_name,

SUM(oi.quantity*oi.unit_price) AS product_sales

FROM products p JOIN order_items oi ON p.product_id=oi.product_id

GROUP BY p.product_id,p.product_name

ORDER BY product_sales DESC

LIMIT 5;
```

24. Sales by Category

```
SELECT p.category,

SUM(oi.quantity*oi.unit_price) AS category_sales

FROM products p JOIN order_items oi ON p.product_id=oi.product_id

GROUP BY p.category

ORDER BY category_sales DESC;
```

25. Top Customers by Sales

```
SELECT c.name,SUM(o.total_amount) AS sales

FROM customers c JOIN orders o ON c.customer_id=o.customer_id

GROUP BY c.customer_id,c.name

ORDER BY sales DESC

LIMIT 5;
```

26. Orders by Status

```
SELECT status,COUNT(*) AS order_count

FROM orders GROUP BY status ORDER BY order_count DESC;
```

27. Payments by Method / Status

```
-- The requested table structure contains payment_status, not payment_method.

-- Therefore this valid analysis uses payment status:

SELECT payment_status,COUNT(*) AS payment_count,

SUM(amount) AS payment_amount

FROM payments

GROUP BY payment_status;
```

28. Repeat Customers / Repeat Purchase Rate

```
WITH customer_orders AS (

SELECT customer_id,COUNT(*) order_count

FROM orders GROUP BY customer_id

)

SELECT COUNT(*) AS repeat_customers

FROM customer_orders

WHERE order_count>1;

SELECT ROUND(

100.0*SUM(CASE WHEN order_count>1 THEN 1 ELSE 0 END)/COUNT(*),2

) AS repeat_purchase_rate

FROM customer_orders;
```

29. Sales by Day

```
SELECT DAYNAME(order_date) AS day_name,

SUM(total_amount) AS sales

FROM orders

GROUP BY DAYNAME(order_date)

ORDER BY sales DESC;
```

30. Validation Checks — Run Before Calling Database Complete

```
-- Exactly 20 customers
```

```
SELECT COUNT(*) AS customers FROM customers;

-- Exactly 20 products

SELECT COUNT(*) AS products FROM products;

-- Exactly 20 orders

SELECT COUNT(*) AS orders FROM orders;

-- Exactly 20 payments

SELECT COUNT(*) AS payments FROM payments;

-- Check duplicate primary keys

SELECT customer_id,COUNT(*) FROM customers GROUP BY customer_id HAVING COUNT(*)>1;

SELECT product_id,COUNT(*) FROM products GROUP BY product_id HAVING COUNT(*)>1;

SELECT order_id,COUNT(*) FROM orders GROUP BY order_id HAVING COUNT(*)>1;

SELECT payment_id,COUNT(*) FROM payments GROUP BY payment_id HAVING COUNT(*)>1;

-- Check orphan order_items

SELECT oi.*

FROM order_items oi

LEFT JOIN orders o ON oi.order_id=o.order_id

LEFT JOIN products p ON oi.product_id=p.product_id

WHERE o.order_id IS NULL OR p.product_id IS NULL;

-- Check orphan payments

SELECT pay.*

FROM payments pay

LEFT JOIN orders o ON pay.order_id=o.order_id

WHERE o.order_id IS NULL;
```